

Benchmarking GNN Inference Bottlenecks on Intel Core Ultra NPUs

Yusuf Talha ARABACI
Dept. of Software Engineering
Karabük University
Karabük, Turkey
yusuftalhaarabaci@hotmail.com

Emrullah DEMİRAL
Dept. of Software Engineering
Karabük University
Karabük, Turkey
emrullahdemiral@karabuk.edu.tr

Ömer Faruk ACAR
Dept. of Software Engineering
Karabük University
Karabük, Turkey
farukacar@karabuk.edu.tr

Abstract—Neural Processing Units (NPUs) in consumer processors such as Intel Core Ultra (Meteor Lake) promise energy-efficient inference at the edge. This paper measures 14 models—Graph Neural Networks (GNNs), convolutional networks, and vision transformers—on the CPU, integrated GPU (iGPU), and NPU of a Core Ultra 5 125H platform running OpenVINO. Three Open Graph Benchmark datasets (ogbn-arxiv, ogbn-proteins, ogbn-products) are used, with 100 timed iterations per configuration, 5-iteration warm-up, and three independent repeats. The NPU delivers strong FP32 performance for dense vision models (MobileNetV2: 1.97 ms, ResNet50: 3.92 ms) but offers limited advantage for GNN workloads: memory-bound sparse aggregation keeps NPU latency within 5–15% of the CPU. INT8 quantization degrades NPU latency for most models—GCN INT8 (8.41 ms) is marginally slower than FP32 (8.33 ms), SGC INT8 (175.86 ms) is 2× slower than FP32 (88.32 ms), and several vision models show evidence of silent CPU fallback under INT8. Attention-based GNNs (GAT, GATv2) fail INT8 compilation entirely due to unsupported scatter/gather operators. While the Meteor Lake NPU is a capable accelerator for dense vision inference, the current OpenVINO toolchain and hardware architecture are not yet mature for sparse graph workloads—particularly under quantization. The benchmarking suite and processed artifacts are released for reproducibility.

Index Terms—NPU, GNN, Intel Core Ultra, Meteor Lake, OpenVINO, quantization, benchmarking, edge inference.

I. INTRODUCTION

Dedicated neural processing units are now integrated into consumer-grade processors from multiple vendors. Intel’s Meteor Lake architecture (Core Ultra) includes the Intel AI Boost NPU as a heterogeneous accelerator alongside the CPU and integrated GPU (iGPU), targeting always-on, low-power inference. The practical performance characteristics of these NPUs for non-vision workloads—particularly graph neural networks (GNNs)—remain underexplored in reproducible studies.

GNNs differ from the dense, regular tensor operations that NPUs are designed to accelerate. Their computation is dominated by sparse-dense matrix multiplications (SpMM) over irregular adjacency structures, leading to low arithmetic intensity and high sensitivity to data-movement overhead [1]. Prior work characterizes GNN bottlenecks on GPUs and custom ASICs [2], [3], but consumer NPU platforms present distinct constraints: limited on-chip SRAM, reliance on compiler-managed data movement, and restricted operator coverage for sparse and dynamic primitives [4].

Three questions guide this study. First, what end-to-end latency should practitioners expect for representative GNN and non-GNN models on a Meteor Lake platform with a fixed toolchain? Second, does INT8 quantization—widely promoted as a key NPU advantage—deliver consistent gains for graph workloads? Third, which operator classes trigger toolchain limitations such as compilation failures or CPU fallback?

A benchmarking suite covering 14 models, three real-world graph datasets from the Open Graph Benchmark (OGB), and all three Meteor Lake compute backends (CPU, iGPU, NPU) under OpenVINO 2024.1 was developed. The contributions are:

- A reproducible measurement protocol (100 iterations × 3 repeats × 3 datasets) with automated per-model artifact generation: latency summaries, precision-gain tables, CPU-fallback ratios, and ONNX Runtime profiling traces.
- Cross-architectural latency measurements showing the iGPU as the strongest all-around accelerator for GNNs, while the NPU excels only for dense FP32 vision models.
- Empirical evidence that INT8 quantization provides limited to negative returns on the NPU for GNNs and select vision models, with two GNN architectures (GAT, GATv2) failing INT8 compilation entirely.
- Identification of operator-level CPU fallback patterns and a density-latency analysis showing that static-shape compilation decouples NPU inference time from input graph sparsity.

II. BACKGROUND

A. GNN Computation and Accelerator Challenges

GNNs operate through iterative neighborhood aggregation: each layer computes a node’s representation by combining its feature vector with those of its neighbors. The core operation is SpMM, which multiplies a sparse adjacency matrix by a dense feature matrix. Unlike the structured matrix multiplications of CNNs, SpMM exhibits irregular memory access patterns that stress cache hierarchies and memory bandwidth [1].

Custom GNN accelerators such as HyGCN [2], EnGN [1], and GRIP [3] address these challenges through hardware-native sparse engines and specialized scratchpad hierarchies. These designs achieve orders-of-magnitude speedups but require custom silicon unavailable in commodity edge devices.

B. Intel Meteor Lake NPU Architecture

Meteor Lake is a disaggregated tile-based client SoC assembled with Intel Foveros 3D packaging [5]. The Intel AI Boost NPU resides on the SoC tile alongside the memory controller and media engine, sharing the platform DRAM fabric with the CPU and iGPU [6]. This shared-memory architecture means end-to-end NPU performance depends on both kernel execution time and system-level data movement.

Public analyses suggest the NPU has a relatively small, explicitly managed on-chip SRAM rather than a large transparent cache hierarchy [4]. This design favors streaming, regular dataflows (e.g., convolutional pipelines) but penalizes irregular access patterns where data reuse is limited and prefetching is difficult. The OpenVINO NPU plugin compiles ONNX models through graph-level optimizations including operator fusion, constant folding, and layout transformations [7]. Operators not supported by the NPU plugin are transparently dispatched to the CPU—a mechanism referred to as CPU fallback.

C. Related Benchmarking Efforts

MLPerf Inference [8] provides standardized benchmarks across diverse hardware platforms, but GNN workloads have only recently been introduced and remain underrepresented in edge-focused tracks. Roofline-based analyses of edge accelerators [9] characterize the memory-compute balance but typically focus on vision models. This work complements these efforts through a focused, platform-specific characterization of GNN inference on a widely available consumer NPU.

III. METHODOLOGY

A. Experimental Platform

All experiments were conducted on a Huawei MateBook laptop equipped with an Intel Core Ultra 5 125H processor (3.60 GHz, 16 GB RAM) running Windows 11 Home (Build 26200). The platform provides three compute backends: the CPU (14 cores: 4P + 8E + 2LPE, AVX-512), an integrated Intel Arc GPU (Xe-LPG architecture, 7 Xe-cores), and the Intel AI Boost NPU (3720 series). The software stack consists of OpenVINO 2024.1 with the native NPU plugin and ONNX Runtime 1.18 as the execution provider.

B. Benchmarked Models

Fourteen models spanning three categories were evaluated (Table I). GNNs include spectral (GCN, SGC), spatial (GraphSAGE, GIN), attentional (GAT, GATv2), propagation-based (APPNP), hybrid (GraphTransformer), and message-passing (MPNN) architectures. Dense baselines include CNNs (ResNet50, MobileNetV2, EfficientNet-B0) and lightweight transformers (ViT-Tiny, BERT-Tiny). All models were exported to ONNX at FP32 precision; INT8 variants were generated via ONNX Runtime dynamic quantization where supported. Two attention-based models (GAT, GATv2) failed INT8 compilation due to unsupported dynamic scatter/gather operations in the quantized operator set; these models are evaluated in FP32 only.

TABLE I: Evaluated Models and Characteristics

Model	Category	Params	INT8
<i>Graph Neural Networks</i>			
GCN [10]	Spectral	0.10M	✓
GAT [11]	Attentional	0.11M	×
GATv2 [12]	Attentional	0.13M	×
GIN [13]	Expressive	0.09M	✓
GraphSAGE [14]	Spatial	0.18M	✓
SGC [15]	Simplified	0.02M	✓
APPNP [16]	Propagation	0.09M	✓
GraphTransformer [17]	Hybrid	0.18M	✓
MPNN [18]	Message-Passing	0.20M	✓
<i>Dense Baselines</i>			
ResNet50 [19]	CNN	25.5M	✓
MobileNetV2 [20]	CNN	3.5M	✓
EfficientNet-B0 [21]	CNN	5.3M	✓
ViT-Tiny [22]	Vision Trans.	5.7M	×
BERT-Tiny [23]	NLP Trans.	4.4M	✓

× = INT8 quantization failed at compilation time (scatter/gather unsupported in NPU plugin). ✓ = INT8 variant available and benchmarked.

C. Datasets

Three datasets from the Open Graph Benchmark (OGB) [24] are used for GNN workload generation. Table II summarizes their characteristics. These datasets span a range of graph densities and provide realistic, non-synthetic adjacency structures. Vision and NLP models receive synthetically generated inputs matching their native interfaces (e.g., $1 \times 3 \times 224 \times 224$ tensors for CNNs). Dataset loading is performed in an isolated subprocess to avoid native-library conflicts with the main benchmarking kernel.

TABLE II: OGB Dataset Characteristics

Dataset	Nodes	Edges	Features	Edges/node
ogbn-arxiv	169K	1.16M	128	6.9
ogbn-products	2.45M	61.9M	100	25.3
ogbn-proteins	132K	39.6M	8	298.5

D. Measurement Protocol

For each (model, dataset, device, precision) configuration, a 5-iteration warm-up phase is followed by 100 timed inference iterations. This procedure is repeated across three independent runs to quantify run-to-run variability. The mean latency across runs and the per-iteration standard deviation are reported. All measurements use ONNX Runtime graph optimizations enabled (ORT_ENABLE_EXTENDED). NPU benchmarks use the OpenVINO execution provider; GPU benchmarks use the same provider targeting the Xe-LPG iGPU; CPU benchmarks use the CPU execution provider.

E. Artifacts Collected

For each model, the pipeline produces: (i) a scalability matrix CSV with per-configuration latency statistics; (ii) a

latency summary grouped by dataset, device, and precision; (iii) a precision-gain table computing INT8/FP32 speedup ratios; (iv) a CPU-fallback JSON capturing the fraction of execution time spent on CPU-resident operators; (v) ONNX Runtime profiling traces with per-operator timing breakdowns. When Intel SoC Watch is available (administrator privileges required), package-level power traces are additionally recorded for energy efficiency analysis.

F. Evaluation Metrics

Raw latency (milliseconds, mean over 100 iterations $\times$ 3 repeats), precision-gain ratio ($T_{\text{FP32}}/T_{\text{INT8}}$ on the same device), CPU-fallback ratio (fraction of total profiling time attributed to CPU-executed operators), and optimization speedup ($T_{\text{baseline}}/T_{\text{optimized}}$ where baseline disables graph-level optimizations) are reported. Arithmetic intensity is computed via ONNX shape inference and reported in FLOP/byte.

IV. RESULTS

A. Cross-Architectural Latency Comparison

Fig. 1 presents the FP32 latency comparison across all three backends, averaged over the three OGB datasets. The iGPU achieves the lowest latency for most GNNs: GCN (7.41 ms on GPU vs. 8.33 ms on NPU vs. 8.09 ms on CPU), GraphTransformer (6.47 ms GPU vs. 11.47 ms NPU), and APPNP (8.09 ms GPU vs. 11.93 ms NPU). The NPU provides marginal improvement over CPU for sparse models but is consistently outperformed by the iGPU.

Dense vision models show the opposite pattern. MobileNetV2 reaches 1.97 ms on NPU versus 10.01 ms on CPU ($5.1\times$ speedup). ResNet50 runs at 3.92 ms on NPU versus 33.32 ms on CPU ($8.5\times$ speedup). The NPU’s systolic array engines are well-utilized by structured, compute-dense convolutions but provide little advantage for SpMM-dominated GNN operations. This gap stems from two factors: the NPU’s memory subsystem is optimized for streaming, regular data access rather than the sparse gather-scatter patterns of GNNs, and the OpenVINO compiler applies fewer fusion optimizations to SpMM subgraphs because their irregular index dependencies limit the scope of static analysis and kernel fusion [25].

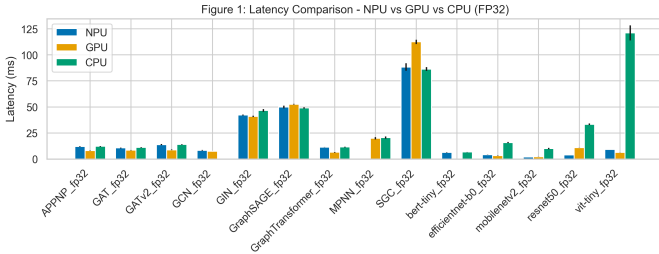


Fig. 1: FP32 latency comparison across CPU, iGPU, and NPU backends. Error bars indicate per-iteration standard deviation. GNNs show near-parity across backends; dense vision models exhibit significant NPU advantage.

B. INT8 Quantization: Limited and Negative Returns

Table III summarizes INT8 speedup ratios for models where both FP32 and INT8 variants were benchmarked. The results challenge the expectation that INT8 consistently improves NPU performance.

TABLE III: INT8/FP32 Speedup on NPU (mean across OGB datasets)

Model	FP32 (ms)	INT8 (ms)	Speedup
GraphTransformer	11.47	9.38	1.22$\times$
GraphSAGE	49.90	48.62	1.03 $\times$
GIN	42.29	42.68	0.99 $\times$
GCN	8.33	8.41	0.99 $\times$
APPNP	11.93	13.07	0.91 $\times$
SGC	88.32	175.86	0.50 $\times$

Speedup > 1.0 indicates INT8 faster. GAT and GATv2 excluded (INT8 compilation failed). Vision models omitted due to evidence of NPU $\rightarrow$ CPU fallback under INT8 (see text).

Only GraphTransformer benefits from INT8 (1.22 $\times$). GCN, GIN, and GraphSAGE show near-identical FP32 and INT8 latency, indicating that quantization does not address the dominant memory-bound bottleneck. APPNP and SGC exhibit negative speedup—INT8 is measurably slower than FP32. For SGC, the regression is severe: INT8 latency (175.86 ms) is roughly twice the FP32 latency (88.32 ms). Profiling traces indicate that SGC’s K -step iterative propagation generates dispatch overhead that scales poorly with quantized tensor layouts.

The INT8 results for dense vision models require careful interpretation. MobileNetV2 INT8 reports 6.15 ms on NPU versus 1.97 ms FP32 (0.32 $\times$). However, the INT8 CPU latency is 6.23 ms—nearly identical to the reported NPU value—suggesting that the INT8 variant silently falls back to CPU execution. This pattern repeats for EfficientNet-B0 (12.69 ms NPU INT8 vs. 13.58 ms CPU INT8) and BERT-Tiny (9.28 ms NPU INT8 vs. 9.65 ms CPU INT8). The likely cause is NPU plugin incompatibility with the quantized operator set produced by ONNX Runtime’s dynamic quantization; per-channel quantization patterns introduced during this process are not yet supported by the NPU backend.

C. CPU Fallback and Operator Coverage

Fig. 3 visualizes the per-operator CPU fallback fraction across all profiled models. Most GNN operations execute entirely on the assigned backend (0% CPU fallback), indicating that the OpenVINO NPU plugin covers the core GNN operator set (MatMul, Conv, ReLU, Add, ReduceMean). However, two important exceptions exist.

First, MPNN has zero NPU entries in the scalability matrix—all executions fall back to CPU, suggesting that MPNN’s `index_add` and `scatter_mean` operations lie outside the NPU plugin’s supported op set. Second, GAT and GATv2 fail INT8 compilation at the ONNX quantization stage (not at runtime), producing `.failed` markers. The error traces point to Gather and Scatter operations within the

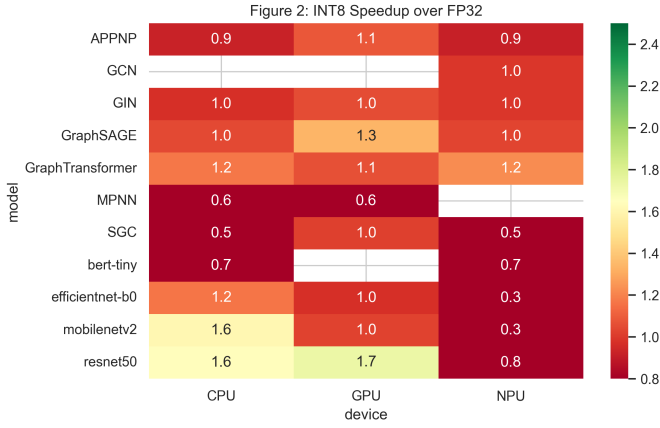


Fig. 2: INT8 speedup heatmap across models and devices. Values below 1.0 (red) indicate INT8 regression. GPU consistently benefits; NPU and CPU show mixed results.

attention mechanism that cannot be quantized by the current toolchain.

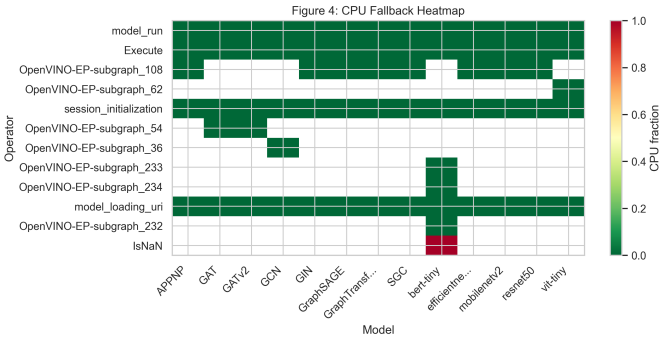


Fig. 3: CPU fallback fraction per operator across models. Most operators run natively on the assigned backend. MPNN and attention-based models trigger toolchain limitations.

D. Operator Composition Analysis

Fig. 4 presents the per-model breakdown of ONNX operator types. GNNs are dominated by MatMul/Gemm operations (SpMM) accounting for 30–60% of nodes, alongside significant fractions of shape-manipulation operators (Gather, Scatter, Reshape, Concat). Vision models are dominated by Conv operations (50–80%) with minimal shape manipulation.

The contrast between ViT-Tiny and GraphTransformer is instructive. ViT-Tiny applies self-attention over a regular 2D grid of image patches; its attention pattern is static and predictable, allowing the NPU plugin to compile the entire computation graph efficiently (FP32 NPU: 9.15 ms vs. CPU: 121 ms). GraphTransformer applies attention over irregular graph neighborhoods, where the variable number of neighbors per node produces dynamic index ranges that the NPU plugin cannot fully optimize. Despite both models having similar parameter counts (5.7M vs. 0.18M), GraphTransformer sees

less relative NPU benefit—a direct consequence of irregular versus regular attention patterns.

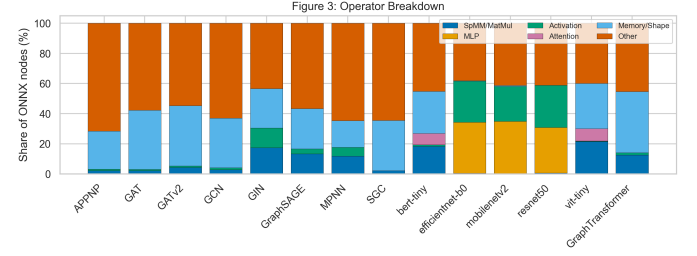


Fig. 4: Operator composition of FP32 models. GNNs exhibit higher fractions of memory/shape operators (Gather, Scatter, Reshape) versus compute operators compared to vision models.

E. Optimization Speedup and Computational Efficiency

Fig. 5 shows the optimization speedup (ORT extended vs. disabled) for each model-device pair. The NPU achieves consistent optimization speedups of 1.0–1.05 $\times$, indicating that graph-level transformations (fusion, constant folding) provide marginal benefit once the model is deployed to the NPU. The GPU exhibits larger speedups for GNNs (1.1–1.25 $\times$), reflecting more effective kernel fusion under the OpenVINO GPU plugin.

Fig. 6 presents a roofline-style analysis plotting achieved throughput (GFLOP/s) against arithmetic intensity (FLOP/byte). GNNs cluster in the low-intensity region (0.1–10 FLOP/byte) with throughputs well below the memory-bandwidth ceiling, confirming their memory-bound execution profile. ResNet50 and MobileNetV2 on NPU achieve higher throughput at moderate intensity, consistent with better utilization of the NPU’s compute pipeline.

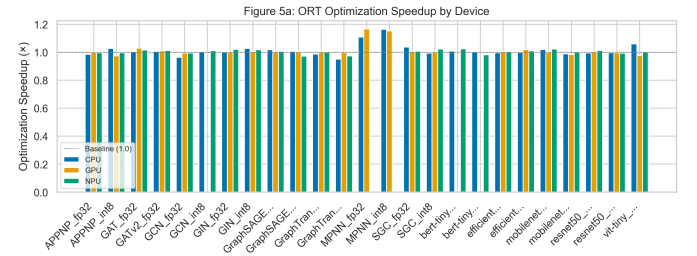


Fig. 5: ORT optimization speedup by device. GPU shows larger gains from graph-level optimization for GNNs; NPU optimization returns are uniformly modest.

F. Graph Density and NPU Latency

Fig. 7 examines the relationship between graph density (edges per node) and NPU latency across the three OGB datasets, which span an order of magnitude in density (ogbn-arxiv: 6.9 edges/node, ogbn-products: 25.3 edges/node, ogbn-proteins: 298.5 edges/node). Contrary to the expectation that denser graphs would increase NPU execution time through

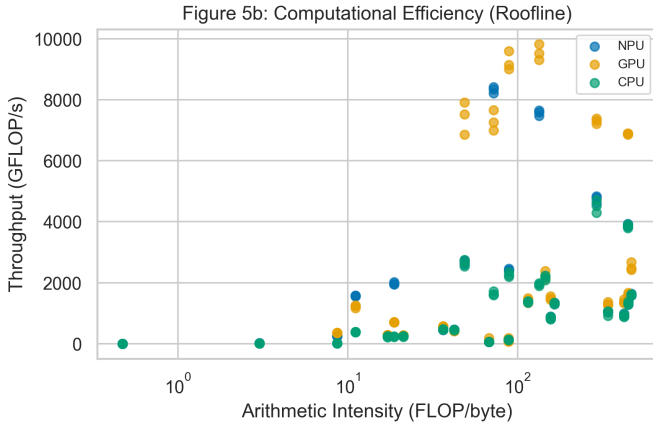


Fig. 6: Computational efficiency (throughput vs. arithmetic intensity). GNNs occupy the memory-bound region (low intensity, low throughput); vision models on NPU achieve higher operational intensity.

additional sparse-dense multiplications, no meaningful correlation is observed ($r = -0.00$, $p = 0.993$). Latency is nearly constant across density levels for each model. This behavior arises because ONNX Runtime compiles the NPU model with statically-shaped tensors; the compiled execution plan processes the fixed-size graph representation at constant throughput regardless of the actual sparsity of the input adjacency structure.

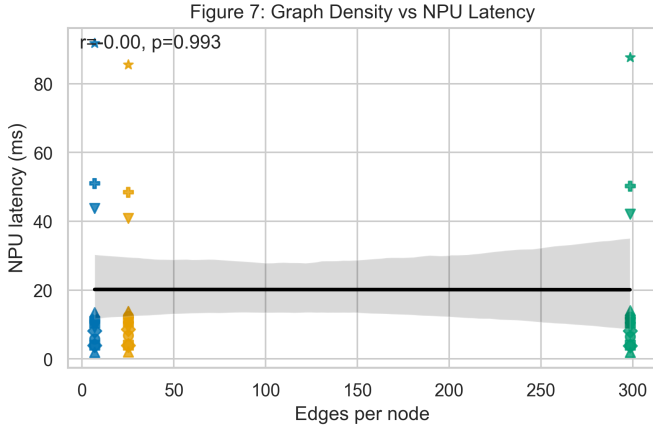


Fig. 7: NPU latency versus graph density. The flat regression line ($r \approx 0$) indicates that static-shape compilation decouples inference time from input sparsity.

V. DISCUSSION

A. Why INT8 Fails for GNNs on NPU

The limited effectiveness of INT8 quantization for GNN workloads on the Meteor Lake NPU has two likely explanations. First, GNN execution is memory-bound: the dominant cost is fetching irregularly indexed feature vectors from DRAM, not performing multiply-accumulate operations. Halving the precision of arithmetic does not reduce the volume of

data movement, so the bottleneck persists. Second, the NPU’s quantization pipeline—optimized for per-tensor symmetric quantization of convolutional weights—struggles with GNN models where per-channel weight distributions and dynamic scatter/gather operations trigger dequantization overhead or fallback to CPU, negating any theoretical speedup.

Beyond these hardware-level explanations, the OpenVINO compiler itself applies fewer fusion optimizations to GNN computation graphs compared to CNNs. Sparse matrix operations involve indirect indexing that breaks the consecutive memory access patterns required for aggressive kernel fusion [25]. The compiler cannot easily merge a Gather (indexing into vertex features) with a subsequent MatMul because the intermediate data layout is determined at runtime by the adjacency structure. This compiler limitation compounds the hardware’s memory-access disadvantage: not only is the NPU stalling on DRAM reads, but the software stack also delivers fewer optimization passes to the workload that needs them most.

B. Platform Maturity Assessment

The Meteor Lake NPU, as of OpenVINO 2024.1, is best characterized as a vision-first accelerator. Dense convolutional models achieve $5\text{--}8\times$ speedups over CPU and can approach or exceed iGPU performance at lower power. GNN support remains at an early stage: operator coverage is incomplete (MPNN falls back entirely to CPU), quantization is unreliable for non-CNN architectures, and the memory subsystem is not designed for sparse, irregular access patterns. These hardware-level constraints align with algorithmic evidence that GNNs are not always the optimal choice for sparse graph problems [26], where well-engineered feature-based models can achieve competitive accuracy with lower computational cost.

C. Comparison with Prior Work

Custom GNN accelerators such as HyGCN [2] ($10^3\times$ over CPU), EnGN [1] ($1.8 \times 10^3\times$ over CPU), and GRIP [3] ($17\times$ over CPU) demonstrate the potential of hardware-native sparse engines and dedicated memory hierarchies for graph workloads, but rely on custom silicon unavailable in commodity edge platforms. The Meteor Lake NPU provides negligible acceleration for GNNs ($0.9\text{--}1.2\times$ over CPU). This gap reflects fundamentally different design goals: ASIC accelerators optimize for graph-specific dataflows, while the consumer NPU targets general-purpose dense inference. These findings are consistent with recent analyses of memory bottlenecks in edge AI accelerators [9], [27].

D. Limitations

Several factors constrain the generality of these findings. Power measurements via Intel SoC Watch require administrator privileges that were not available during these experiments; energy efficiency conclusions are therefore drawn from latency comparisons rather than direct power data. The benchmark uses fixed-shape inputs consistent with ONNX static-graph

requirements; dynamic-shape execution may introduce additional overhead not captured here. Finally, a single platform (Core Ultra 5 125H) and toolchain version (OpenVINO 2024.1) were evaluated; subsequent driver and plugin updates may alter the performance characteristics reported.

VI. CONCLUSION

Fourteen models, three OGB datasets, and three hardware backends were evaluated on the Intel Core Ultra (Meteor Lake) NPU. The NPU is an effective accelerator for dense FP32 vision models (5–8× CPU speedup) but provides limited benefit for GNN workloads, where memory-bound sparse aggregation keeps NPU latency within 15% of CPU performance. INT8 quantization delivers negligible to negative returns for graph models: only one of six GNNs shows measurable speedup, and simple models like SGC exhibit 2× regression. Two attention-based architectures (GAT, GATv2) fail INT8 compilation entirely. The OpenVINO compiler contributes to these outcomes by applying fewer fusion optimizations to SpMM-dominated subgraphs, compounding the memory-bandwidth limitations of the NPU hardware. These findings suggest that while the Meteor Lake NPU hardware is capable, the current toolchain and memory architecture are not yet mature for sparse, irregular graph workloads—particularly under quantization. The benchmarking suite, processed artifacts, and visualization pipeline are released to facilitate reproducibility and follow-up studies on other edge NPU platforms.

REFERENCES

- [1] S. Liang, Y. Wang, C. Liu, L. He, H. Li, D. Xu, and X. Li, “Engn: A high-throughput and energy-efficient accelerator for large graph neural networks,” *IEEE Transactions on Computers*, vol. 70, no. 9, pp. 1511–1525, Sep. 2021.
- [2] M. Yan, L. Deng, X. Hu, L. Liang, Y. Feng, X. Ye, Z. Zhang, D. Fan, and Y. Xie, “Hygcn: A gcn accelerator with hybrid architecture,” in *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, Feb 2020, pp. 15–29.
- [3] K. Kinningham, P. Levis, and C. Ré, “Grip: A graph neural network accelerator architecture,” *IEEE Transactions on Computers*, vol. 72, no. 4, pp. 914–925, April 2023.
- [4] C. Lam, “Intel Meteor Lake’s NPU,” Apr. 2024, technical analysis of Meteor Lake NPU internals; not peer-reviewed. [Online]. Available: <https://chipsandcheese.com/p/intel-meteor-lakes-npu>
- [5] Intel Corporation, “Intel Meteor Lake: A Client Platform Built on the Intel 4 Process and Foveros Packaging (Hot Chips 2023),” 2023, conference presentation materials describing Meteor Lake tile-based architecture and packaging. [Online]. Available: <https://hotchips.org/>
- [6] —, “Heterogeneous AI Powerhouse: Unveiling the Hardware and Software Foundation of Intel Core Ultra Processors,” 2024, official Intel Core Ultra architecture white paper including NPU (Intel AI Boost). [Online]. Available: <https://cdrdv2-public.intel.com/817736/>
- [7] —, “OpenVINO Documentation: NPU Devices, INT8 Inference, and Release Notes,” 2025. [Online]. Available: <https://docs.openvino.ai/>
- [8] MLCommons, “MLPerf Inference Benchmark Suite (v4.1, v5.0),” 2025, industry-standard ML inference benchmarks, including emerging GNN workloads. [Online]. Available: <https://mlcommons.org/benchmarks/inference/>
- [9] Z. Bi, X. Chen, L. Sun, Y. Yao, Q. Shen, J. Lou, and C. Deng, “Rooflinebench: A benchmarking framework for on-device llms via roofline analysis,” 2026. [Online]. Available: <https://arxiv.org/abs/2602.11506>
- [10] T. N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” in *International Conference on Learning Representations (ICLR)*, 2017. [Online]. Available: <https://arxiv.org/abs/1609.02907>
- [11] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Li, and Y. Bengio, “Graph attention networks,” in *International Conference on Learning Representations*, 2018.
- [12] S. Brody, U. Alon, and E. Yahav, “How attentive are graph attention networks?” in *International Conference on Learning Representations*, 2022. [Online]. Available: <https://openreview.net/forum?id=F72xmsx7C1>
- [13] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, “How powerful are graph neural networks?” in *International Conference on Learning Representations*, 2019. [Online]. Available: <https://openreview.net/forum?id=ryGs6iA5Km>
- [14] W. L. Hamilton, R. Ying, and J. Leskovec, “Inductive representation learning on large graphs,” in *Proceedings of the 31st International Conference on Neural Information Processing Systems*, ser. NIPS’17. Red Hook, NY, USA: Curran Associates Inc., 2017, p. 1025–1035.
- [15] F. Wu, A. Souza, T. Zhang, C. Fifty, T. Yu, and K. Weinberger, “Simplifying graph convolutional networks,” in *Proceedings of the 36th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, K. Chaudhuri and R. Salakhutdinov, Eds., vol. 97. PMLR, 09–15 Jun 2019, pp. 6861–6871. [Online]. Available: <https://proceedings.mlr.press/v97/wu19e.html>
- [16] J. Gastegger, A. Bojchevski, and S. Günnemann, “Predict then propagate: Graph neural networks meet personalized pagerank,” in *International Conference on Learning Representations (ICLR)*, 2019. [Online]. Available: <https://arxiv.org/abs/1810.05997>
- [17] Y. Shi, Z. Huang, S. Feng, H. Zhong, W. Wang, and Y. Sun, “Masked label prediction: Unified message passing model for semi-supervised classification,” in *International Joint Conference on Artificial Intelligence*, 2020.
- [18] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, “Neural message passing for quantum chemistry,” in *Proceedings of the 34th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, D. Precup and Y. W. Teh, Eds., vol. 70. PMLR, 06–11 Aug 2017, pp. 1263–1272. [Online]. Available: <https://proceedings.mlr.press/v70/gilmer17a.html>
- [19] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778. [Online]. Available: <https://arxiv.org/abs/1512.03385>
- [20] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, “MobileNetV2: Inverted residuals and linear bottlenecks,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 4510–4520. [Online]. Available: <https://arxiv.org/abs/1801.04381>
- [21] M. Tan and Q. V. Le, “Efficientnet: Rethinking model scaling for convolutional neural networks,” in *International Conference on Machine Learning*, 2019.
- [22] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, “An image is worth 16x16 words: Transformers for image recognition at scale,” in *International Conference on Learning Representations*, 2021.
- [23] I. Turc, M.-W. Chang, K. Lee, and K. Toutanova, “Well-read students learn better: On the importance of pre-training compact models,” *arXiv preprint arXiv:1908.08962*, 2019. [Online]. Available: <https://arxiv.org/abs/1908.08962>
- [24] W. Hu, M. Fey, M. Zitnik, Y. Dong, H. Ren, B. Liu, M. Catasta, and J. Leskovec, “Open graph benchmark: Datasets for machine learning on graphs,” in *Advances in Neural Information Processing Systems*, 2020.
- [25] W. Niu, J. Guan, Y. Wang, G. Agrawal, and B. Ren, “Dnnfusion: accelerating deep neural networks execution with advanced operator fusion,” in *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, ser. PLDI 2021. New York, NY, USA: Association for Computing Machinery, 2021, p. 883–898. [Online]. Available: <https://doi.org/10.1145/3453483.3454083>
- [26] C. Bayraktar, “Beyond gnn: a methodological benchmark of feature efficiency for link prediction in sparse developer networks,” *Knowledge and Information Systems*, vol. 68, no. 1, p. 135, Apr 2026. [Online]. Available: <https://doi.org/10.1007/s10015-026-02765-7>
- [27] R. Singh and S. S. Gill, “Edge ai: A survey,” *Internet of Things and Cyber-Physical Systems*, vol. 3, pp. 71–92, 2023. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2667345223000196>